Image Processing - Exercise 1

Itzhak Fagebaume, itzhakfage,666225701

Introduction

a) In this exercise, the goal is to detect scene cuts in videos by identifying the frames where a transition occurs between two different scenes. The main technique used is histogram comparison, employing two variants: the standard histogram distance and the cumulative histogram distance, specifically calculating distances between histograms of consecutive grayscale frames to detect significant changes indicative of a scene cut.

(b) The two categories of videos differ in their transition characteristics. Category 1 videos feature abrupt scene changes with distinct content shifts, making them suitable for standard histogram distance methods. Category 2 videos, however, include changes in quality, such as sudden darkening or obscurity, which can introduce gradual brightness shifts that mimic scene cuts in basic histogram comparisons, requiring a more robust approach like cumulative histogram distances to differentiate true scene cuts from lighting variations.

Algorithm

a) The algorithm for scene cut detection involves the following conceptual steps: 1) Load the video and extract frames, converting each to grayscale. 2) Compute the histogram for each grayscale frame. 3) Calculate the L1 distance between histograms of consecutive frames. 4) Identify the pair of frames with the maximum distance as the scene cut point. 5) Return the indices of the last frame of the first scene and the first frame of the second scene.

(b) For the second category, the algorithm is modified to use cumulative histograms instead of standard ones, specifically by inserting a step between calculating the histograms (step 3) and identifying the maximum distance (step 4), where we compute the cumulative distribution function (CDF) for each frame's histogram and then calculate the L1 distance between these CDFs. This adjustment accounts for overall distribution shifts rather than bin-specific differences, making it less sensitive to gradual changes like darkening, which affect intensity levels uniformly but not the cumulative distribution

as drastically for non-scene-cut frames. Indeed, The standard histogram measures how many pixels belong to each intensity bin, making it very sensitive to uniform changes in brightness. In videos where illumination gradually varies, this causes large histogram differences even when the scene does not actually change. The cumulative histogram (CDF), however, measures the global shape of the distribution. Because gradual brightness changes shift pixel values without significantly altering the overall cumulative distribution, the CDF remains stable within the same scene. Therefore, computing distances between CDFs is more robust for category 2 videos, where lighting changes occur continuously but scene boundaries remain distinct. For exemple, A gradual change in brightness produces nearly the same CDF curve, only slightly shifted, which means the CDF distance stays small.

Implementation Details

(a) The implementation starts with a load function that uses cv2.VideoCapture to read the video file, iterating through frames, converting each to grayscale with cv2.cvtColor, and storing them in a list using NumPy arrays for efficient handling. For detection, separate functions detect_scene_1 and detect_scene_2 process the frames: they compute histograms for each frame using a helper function that wraps cv2.calcHist to get 256-bin grayscale histograms, reshapes them to 1D NumPy arrays, and converts to float for precision. Distances are calculated via a custom helper function compute_distance, which uses NumPy's np.abs and np.sum to compute the L1 norm (sum of absolute differences) between two histograms or CDFs. For category 2, an additional compute_cdf helper uses np.cumsum to generate cumulative sums from the histograms before distance calculation. The index of the maximum distance is found with np.argmax on the list of distances, and the cut frames are returned as (max_idx, max_idx + 1).

(b) I implemented several parts from scratch using NumPy for array operations: the distance calculation (np.sum(np.abs(diff))), CDF computation (np.cumsum), distance list accumulation, and max index finding (np.argmax). Library functionalities include cv2 for video loading (VideoCapture), frame reading, grayscale conversion (cvtColor), and histogram calculation (calcHist, with parameters for single channel, 256 bins, and full range). Matplotlib was used for plotting distance graphs in the visualization helpers. I chose cv2 for its optimized computer vision tools, essential for video and image processing; NumPy for fast numerical computations on arrays, and Matplotlib for generating plots to visualize results.

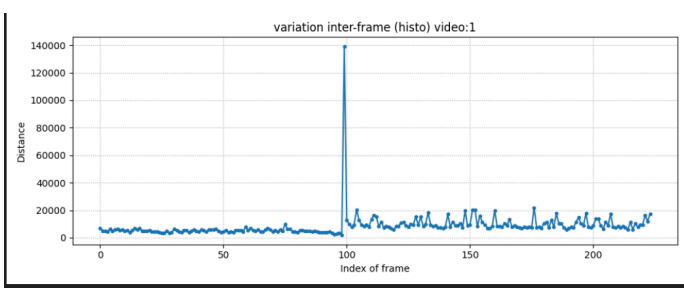
(c) The algorithm uses fixed hyper-parameters: 256 bins for histograms to cover the full 8-bit grayscale range, ensuring detailed intensity distribution capture. No explicit thresholds were used; instead, the maximum distance via argmax assumes a single prominent cut per video. Choices include using L1 distance for its simplicity and effectiveness in detecting bin-wise changes, and applying cumulative sums without normalization to preserve absolute count differences, which empirically worked well for distinguishing cuts from quality shifts.

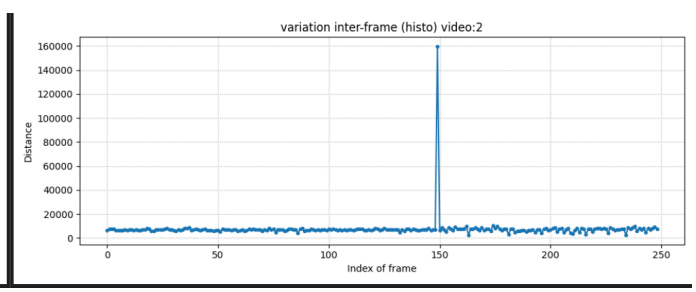
(d) Challenges included selecting and utilizing the appropriate library functions effectively to ensure accuracy and efficiency. For category 2, initial tests with non-cumulative distances detected false positives from darkening; implementing CDFs resolved this by accumulating differences, making uniform brightness shifts less impactful. Debugging involved adding visualization functions to plot distances, revealing patterns like multiple peaks in category 2 without CDFs.

Category 1 Results

(a) For the videos in category 1, the algorithm detected the scene cuts accurately. For example, in video1_category1.mp4, the cut was between frames 99 and 100; in video2_category1.mp4, between 149 and 150.

(b) Figures supporting the detection include plots of histogram distances between consecutive frames, showing a clear peak at the cut point





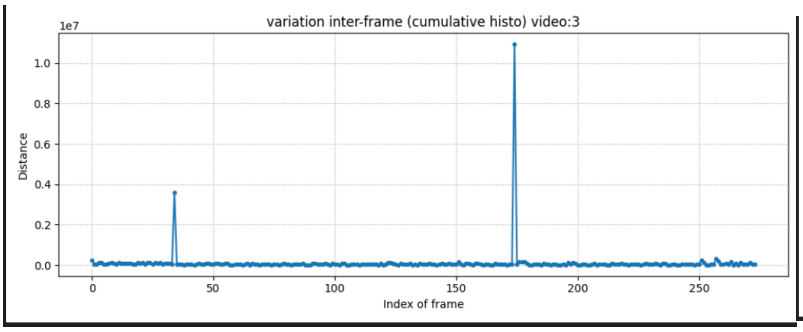
Category 2 Results

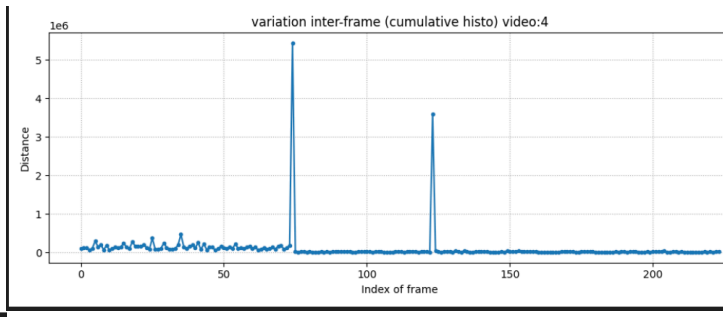
(a) For the videos in category 2, the algorithm detected the scene cuts accurately. For example, in video3_category2.mp4, the cut was between frames 174 and 175; in video4_category2.mp4, between 74 and 75.

(b) The videos in category 2 differ from category 1 by having quality changes, such as abrupt darkening in parts of the scene without changing the content , and also in pixel intensity ranges where initial frames reach up to around 230 rather than the full 255. I

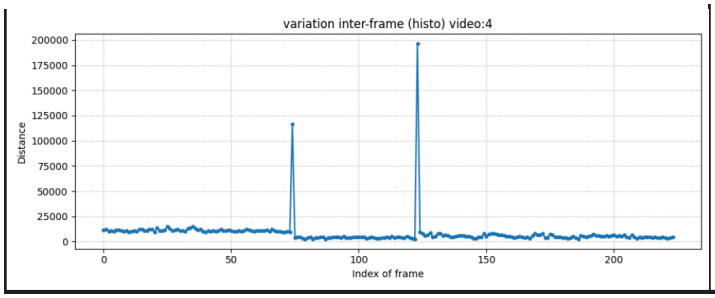
noticed these differences by visually inspecting frames and observing that histogram distances spiked during darkening events in category 2, leading to false positives with the basic algorithm, and print the max intensity pixel in each frame.

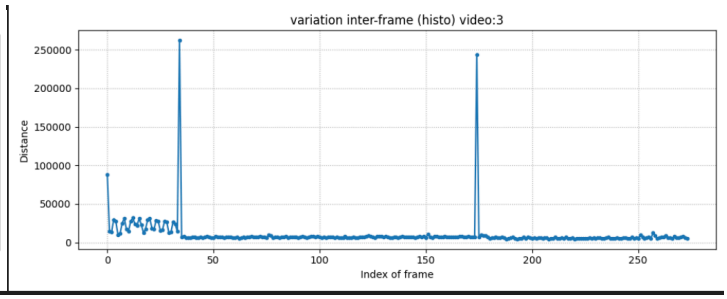
(c) Figures include cumulative histogram distance between consecutive frames, showing a most signifiant peak at the cut point.





(d) Visualizations demonstrating why the category 1 algorithm failed include distance plots without cumulatives (Figure 6), where multiple peaks appear due to darkening and the most significant peak is misplaced, incorrectly identifying non-cut frames as transitions, unlike the clear peak in cumulative versions.





Conclusion

Key findings include the effectiveness of histogram distances for abrupt scene cuts in category 1 and the necessity of cumulative histograms for handling quality variations in category 2. This exercise highlighted how lighting changes can confound basic methods, emphasizing the importance of robust distribution comparisons in image processing tasks. Overall, the approach successfully detected cuts in all provided videos, providing insights into adapting algorithms for different video characteristics.